
TD Terminaison - Correction

Question 1 : Soit l'algorithme suivant :

```
1 def f(a):
2     while a != 0:
3         a = a-1
4     return a
```

1. Cet algorithme termine-t-il pour toute entrée ? Si non, donnez un contre-exemple.
2. Si non, que faudrait-il rajouter à ce code pour être sûr qu'il termine ?

Correction Q1.

1. Non, il ne termine pas si $a < 0$. Contre-exemple : $a = -1$. La condition $a \neq 0$ reste indéfiniment vraie.
2. Il suffit de vérifier que $a \geq 0$ en pré-condition (ajout d'une assertion `assert a >= 0`, ou changement de la boucle en `while a > 0`).

Question 2 : Prouvez la terminaison de la fonction suivante à l'aide d'un variant de boucle :

```
1 def somme(n):
2     res = 0
3     i = 0
4     while i < n:
5         res += i
6         i += 1
7     return res
```

Correction Q2. On peut choisir la fonction $n - i$.

- À chaque itération de la boucle, i augmente de 1, donc $n - i$ diminue strictement de 1.
- Tant qu'on rentre dans la boucle on a $i < n$, donc $n - i > 0$.
- C'est un variant de boucle à valeurs dans $\mathbb{N}$, la boucle termine.

Question 3 : On s'intéresse au calcul de la factorielle de manière récursive :

```
1 def factorielle(n):
2     if n == 0:
3         return 1
4     return n * factorielle(n - 1)
```

1. Donnez une pré-condition pour que la fonction termine sans erreur.
2. Prouvez la terminaison de cet algorithme à l'aide d'un variant dans $\mathbb{N}$.

Correction Q3.

1. Il faut que n soit un entier naturel ($n \geq 0$). Si $n < 0$, on plongerait dans des appels récursifs infinis vers les négatifs.
2. Le variant choisi est simplement n . À chaque appel récursif, la valeur passée est $n - 1$, donc le variant décroît strictement de 1. L'entier reste positif ou nul jusqu'à atteindre la condition d'arrêt $n = 0$.

Question 4 : On souhaite calculer x^n de manière récursive avec l'algorithme « d'exponentiation rapide » suivant :

```
1 def puissance(x, n):
2     if n == 0:
3         return 1
4     if n % 2 == 0:
5         return puissance(x * x, n // 2)
6     else:
7         return x * puissance(x, n - 1)
```

1. Donner les spécifications de la fonction.
2. Prouvez la terminaison de cet algorithme à l'aide d'un variant bien choisi. Vous vérifierez que chaque branche récursive fait bien décroître et validera la terminaison.

Correction Q4.

1.

Entrées : $x \in \mathbb{R}$, $n \in \mathbb{N}$

Sortie : x^n .

2. On choisit le variant n . Soit $n > 0$:

- Si n est pair, le prochain appel se fait avec $n' = \lfloor \frac{n}{2} \rfloor$. Puisque $n \geq 2$, on a strictement $\lfloor \frac{n}{2} \rfloor < n$. L'entier décroît.
- Si n est impair, le prochain appel se fait avec $n' = n - 1$. Puisque $n \geq 1$, on a strictement $n - 1 < n$. L'entier décroît.

Dans tous les cas, n décroît strictement jusqu'à 0, l'algorithme termine donc pour tout $n \in \mathbb{N}$.

Question 5 : Soit l'algorithme de division euclidienne suivant :

```
1 def div_entiere(a, b):
2     q = 0
3     while a >= b:
4         a = a - b
5         q = q + 1
6     return q
```

1. Que se passe-t-il si b vaut 0 ?
2. Donnez des pré-conditions sur a et b pour que cet algorithme termine.
3. Justifiez la terminaison à l'aide d'un variant de boucle.

Correction Q5.

1. Si $b = 0$, et que $a \geq 0$, alors $a - b = a$. La valeur de a ne change pas et la boucle tourne à l'infini.
2. Pré-conditions : $a \geq 0$ et $b > 0$.
3. Variant : a .
 - Tant que la boucle s'exécute $a \geq b > 0$, on a $a > 0$.
 - À chaque itération, a devient $a - b$. Puisque $b > 0$, a décroît strictement.
 - Le variant est à valeurs dans $\mathbb{N}$ et décroît strictement, la boucle termine.

Question 6 : Indiquez si les algorithmes suivants terminent pour tout $n \in \mathbb{N}$. Si ce n'est pas le cas, donnez l'ensemble des valeurs de n pour lesquelles l'algorithme ne termine pas.

Algorithme 1 :

```
1 def algo_1(n):
2     while n != 0:
3         n = n - 2
4     return n
```

Algorithme 2 :

```
1 def algo_2(n):
2     while n > 0:
3         n = n + 1
4     return n
```

Algorithme 3 :

```
1 def algo_3(n):
2     while n > 1:
3         if n % 2 == 0:
4             n = n // 2
5         else:
6             n = 3 * n + 1
7     return n
```

Correction Q6.

- **Algorithme 1** : Ne termine pas. L'ensemble des valeurs de n pour lesquelles l'algo boucle à l'infini est donc si n est impair ou $n < 0$.
- **Algorithme 2** : Ne termine pas. L'ensemble des valeurs pour lesquelles l'algo ne termine pas est $n > 0$.
- **Algorithme 3** : Cet algorithme correspond à la suite de Syracuse. On voit que la valeur de n diminue parfois (division par 2) et augmente parfois (multipliée par 3). On ne peut donc pas trouver de fonction variant strictement décroissante simple. Prouver sa terminaison pour tout $n > 0$ est par ailleurs un problème ouvert en mathématiques.

Question 7 : Considérez le programme suivant avec deux variables :

```
1 def mystere(m, n):
2     while m > 0 or n > 0:
3         if m > 0:
4             m = m - 1
5             n = n + 10
6         else:
7             n = n - 1
```

Prouvez que cet algorithme termine pour tout couple d'entiers naturels (m, n) . *Indication : il n'est pas possible de trouver une simple fonction variant à valeurs dans $\mathbb{N}$ qui décroît strictement à chaque itération. Cherchez plutôt à utiliser un tuple de variables et un ordre bien fondé dessus (par exemple l'ordre lexicographique).*

Correction Q7. On choisit le tuple (m, n) doté de l'ordre lexicographique, défini sur $\mathbb{N} \times \mathbb{N}$. L'ordre bien fondé lexicographique stipule que $(m', n') < (m, n)$ si :

- $m' < m$
- ou bien $(m' = m \text{ et } n' < n)$.

À chaque itération, l'une des deux branches conditionnelles est exécutée :

1. Si $m > 0$: le nouveau tuple vaut $(m - 1, n + 10)$. Puisque $m - 1 < m$, on a $(m - 1, n + 10) < (m, n)$ au sens lexicographique, peu importe l'augmentation de n .
2. Si $m \leq 0$: on a nécessairement $m = 0$ et $n > 0$ pour continuer la boucle. Le nouveau tuple vaut $(m, n - 1)$. On a donc $m' = m$ et $n - 1 < n$. Ainsi $(m, n - 1) < (m, n)$ au sens lexicographique.

Dans les deux cas, le tuple décroît strictement pour l'ordre lexicographique. L'ensemble $\left(\mathbb{N} \times \mathbb{N}, <_{\{\text{lex}\}}\right)$ étant bien fondé (il n'existe pas de suite infinie strictement décroissante), l'algorithme termine à tous les coups.